Week 9: Automate the Running of a Containerized Application

Kubernetes is an open-source platform for **managing containerized workloads and services**. It provides a **robust framework for automating deployments, scaling, and managing containerized applications**

Prerequisites

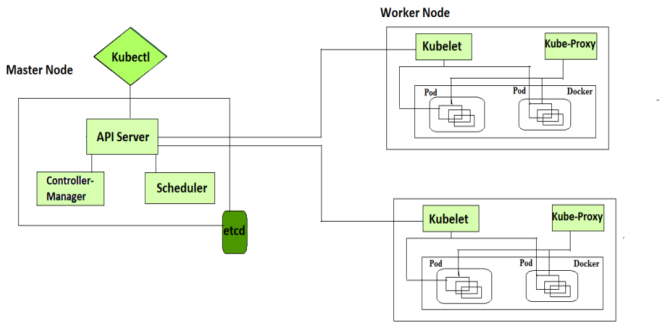
following prerequisites are required:

- **Kubernetes Cluster**: A running Kubernetes cluster. You can set up a local cluster using Minikube or use a managed service like Google Kubernetes Engine (GKE).
- **kubectl**: The command-line tool for interacting with your Kubernetes cluster.
- **Docker**: Installed and running on your machine.
- **Containerized Application**: Your application packaged into a Docker image.

Kubernetes Cluster Architecture

Kubernetes comes with a client-server architecture. It consists of master and worker nodes

The master node, contains the components such as API Server, controller manager, scheduler, and etcddatabase for stage storage. kubelet to communicate with the master, the kube-proxy for networking, and a container runtime such as Docker to manage containers.



Kubernetes Components

Kubernetes is composed of a number of components, each of which plays a specific role in the overall system. These components can be divided into two categories:

- **nodes:** Each Kubernetes cluster requires at least one worker node, which is a collection of worker machines that make up the nodes where our container will be deployed.
- **Control plane:** The worker nodes and any pods contained within them will be under the control plane.

Control Plane Components

It is basically a collection of various components that help us in managing the overall health of a cluster. For example, if you want to set up new pods, destroy pods, scale pods, etc. Basically, 4 services run on Control Plane**:**

1. Kube-API server

2. Kube-Scheduler

3. Kube-Controller-Manager

4. etcd

Node Components

These are the nodes where the actual work happens. Each Node can have multiple pods and pods have containers running inside them.

1. Container runtime

2. kubelet

3. kube-proxy

Test.java

```
import java.io.IOException;

import java.net.InetSocketAddress;

import com.sun.net.httpserver.HttpServer;
```

```
public class Test {
   public static void main(String[] args) throws IOException
{
      int port = 8080; // Port on which the server will listen
      System.out.println("Starting server on port " + port);
      HttpServer server = HttpServer.create(new InetSocketAddress(port), 0);
      server.createContext("/", (exchange -> {
         String response = "Hello, Kubernetes!";
         exchange.sendResponseHeaders(200, response.getBytes().length);
         exchange.getResponseBody().write(response.getBytes());
         exchange.close();
      }));
      server.setExecutor(null);
      server.start();
      System.out.println("Server is running at http://localhost:" + port);
   }
}
```

Step 1: Create a Docker Image

First, create a Docker image for your application. This involves writing a Dockerfile that specifies the base image, copies the application code, and sets the command to run the application.

Dockerfile

```
# Use OpenJDK base image
FROM openjdk:17-alpine
```

```
# Set working directory

WORKDIR /app

# Copy all files to the working directory

COPY . /app

# Compile the Java application

RUN javac Test.java

# Expose port 8080 for the application

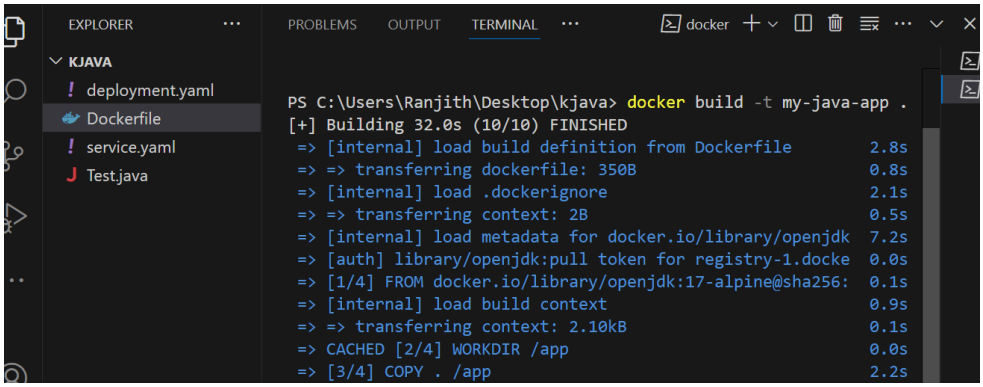
EXPOSE 8080

# Run the Java application

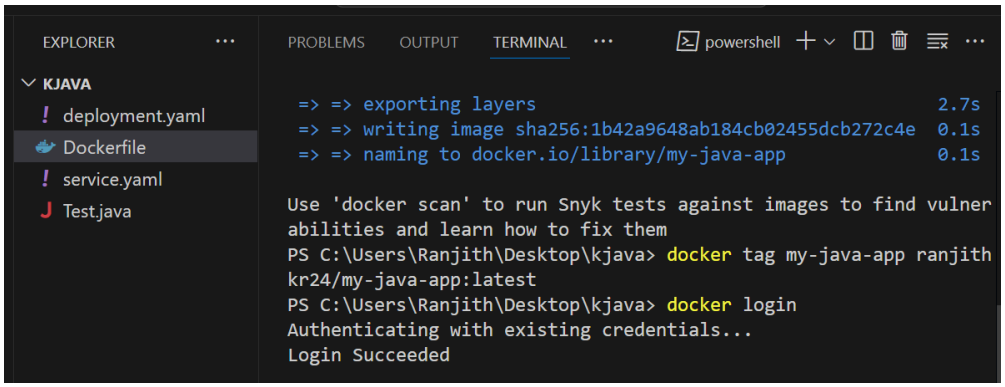
CMD ["java", "Test"]
```

Build the Docker image using the Dockerfile:

- **docker build -t my-java-app .**



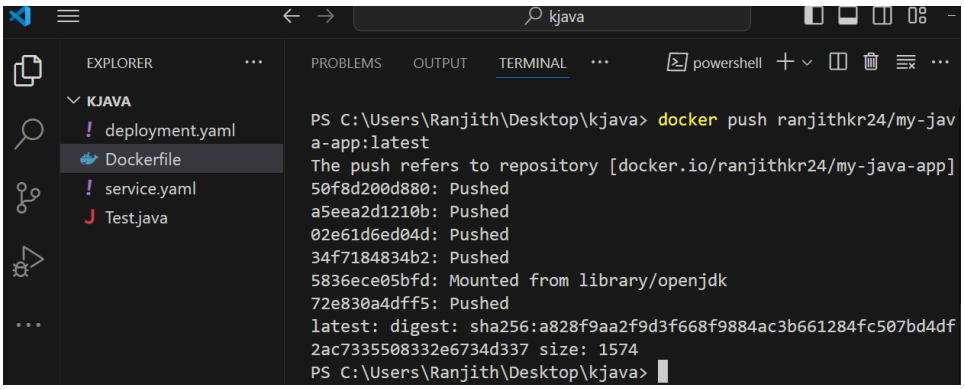
Tagging the image and login into dockerhub



Step 2: Push the Docker Image to a Registry

Push the Docker image to a container registry like Docker Hub

docker tag my-java-app:latest<your-docker-hub-username>/my-java-app:latest
docker push <your-docker-hub-username>/my-java-app:latest



Step 3: Create a Kubernetes Deployment

A Kubernetes Deployment is a resource that manages the rollout of new versions of an application. It ensures that a specified number of replicas (i.e., copies) of your application are running at any given time.

Create a YAML file named deployment.yaml with the following content:

apiVersion: apps/v1

kind: Deployment

metadata:

```
  name: my-java-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-java-app
  template:
    metadata:
      labels:
        app: my-java-app
    spec:
      containers:
        - name: my-java-app
          image: ranjithkr24/my-java-app:latest
          ports:
            - containerPort: 8080
          env:
            - name: PORT
              value: "8080"
```

Apply the deployment configuration to your Kubernetes cluster:

kubectl apply -f deployment.yaml

Step 4: Create a Kubernetes Service

A Kubernetes Service provides a network identity and load balancing for accessing your application. Create a YAML file named service.yaml with the following content:

apiVersion: v1

kind: Service

metadata:

name: my-java-app-service

spec:

selector:

app: my-java-app

ports:

- protocol: TCP

port: 80 # Port exposed inside the cluster

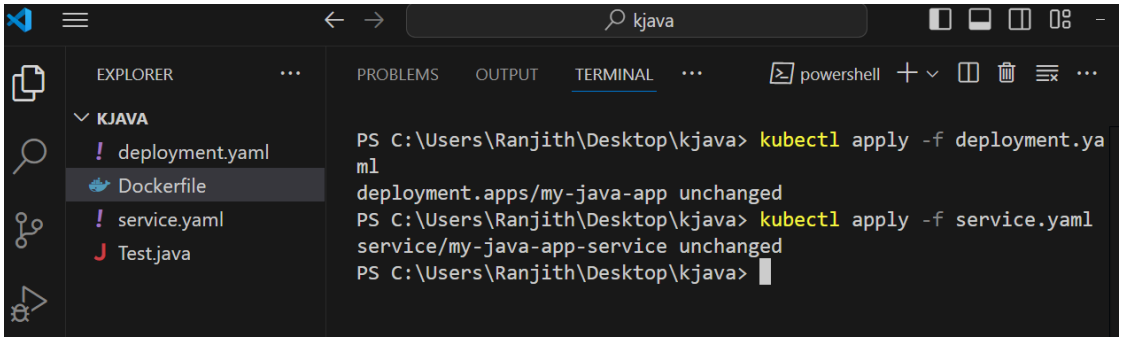
targetPort: 8080 # Port your Java app is running on

nodePort: 30007 # External port

type: NodePort

Apply the service configuration to your Kubernetes cluster:

kubectl apply -f service.yaml



Step 5: Verify the Deployment

Check the status of your deployment and service:

kubectl get deployments

```
PS C:\Users\Ranjith\Desktop\kjava> kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
deployment1        0/3     3            0           7d5h
java-app           0/3     3            0           45h
java-app1          0/3     1            0           45h
java-deployment    0/3     3            0           7d14h
my-java-app        2/3     3            2           46h
myjava1-deploy     0/3     3            0           8d
PS C:\Users\Ranjith\Desktop\kjava> kubectl get deployments
NAME               READY   UP-TO-DATE   AVAILABLE   AGE
deployment1        0/3     3            0           7d5h
java-app           0/3     3            0           45h
java-app1          0/3     1            0           45h
java-deployment    0/3     3            0           7d14h
my-java-app        3/3     3            3           46h
myjava1-deploy     0/3     3            0           8d
```

kubectl get pods

```
Dockerfile
! service.yaml
J Test.java
(4m1s ago)     7d14h
java-deployment-647885494-v2fv7   0/1     CrashLoopBackOff   239
(3m50s ago)    7d14h
my-java-app-5d79d95f68-bwhm4      1/1     Running            2 (4
2m ago)        46h
my-java-app-5d79d95f68-bz42k      1/1     Running            3
               46h
my-java-app-5d79d95f68-mcjjr      1/1     Running            2 (4
2m ago)        46h
```

kubectl get svc

```
KJAVA
! deployment.yaml
Dockerfile
! service.yaml
J Test.java

PS C:\Users\Ranjith\Desktop\kjava> kubectl get svc
NAME                 TYPE           CLUSTER-IP       EXTERNAL-IP
   PORT(S)        AGE
java-app-service     NodePort       10.96.60.200     <none>
   80:30008/TCP   45h
java-app1-service    NodePort       10.107.54.172    <none>
   80:30010/TCP   45h
java-service         LoadBalancer   10.96.207.224    localhost
   80:32029/TCP   7d15h
kubernetes           ClusterIP      10.96.0.1        <none>
   443/TCP        11d
my-java-app-service  NodePort       10.106.150.206   <none>
   80:30007/TCP   46h
myjava1-service      LoadBalancer   10.103.169.241   localhost
   80:31676/TCP   8d
```

Step 6: Access the Application

To access your application, you need to get the external IP address of the service:

Open a web browser and navigate to the external IP address to access your application.

Type: http://localhost:port number of the application service

Type in the browser -- http://localhost:30007 to access the application for the above example

